UNIVERSIDADE ESTADUAL PAULISTA
"JÚLIO DE MESQUITA FILHO"
Campus de São José dos Campos
Instituto de Ciência e Tecnologia

Ensembles

Bayes, Voting, Bagging, Stacking, AdaBoost and Random Forest

Prof. Dr. Rogério Galante Negri

Agenda

1. Let's combine classifiers
2. Combination & Bayes
3. Majority Voting
4. Stacking
5. Bagging
6. AdaBoost/SAMME
7. Random Forest

Classifier Combination

- So far, we have seen several classification methods.
- The arsenal can be further expanded by combining the methods we already know!
- We can achieve more robust results with lower error rates.
- How to proceed?
  - Combine classifiers operating on the same or different feature spaces.
  - Consider combinations applied to data, features, or classifiers.
  - Derive combination schemes that may or may not require training.
  - Generate combinations involving identical or distinct methods.
  - Obtain models with optimization/parameterization performed jointly or separately for each component.

Agenda

1 Let's combine classifiers

2 Combination & Bayes

3 Majority Voting

4 Stacking

5 Bagging

6 AdaBoost/SAMME

7 Random Forest

Combination involving the Bayes concept

- Based on the Bayes classification rule, we decide in favor of class ω_j, instead of any other class ω_k, when:

$$p(\omega_j|\mathbf{x}^{(1)},\ldots,\mathbf{x}^{(L)}) > p(\omega_k|\mathbf{x}^{(1)},\ldots,\mathbf{x}^{(L)})), \tag{1}$$

  for $k = 1,\ldots,c;\ k \neq j$.
- According to Bayes' Theorem, the above relation can be rewritten as:

$$p(\mathbf{x}^{(1)},\ldots,\mathbf{x}^{(L)}|\omega_j)p(\omega_j) > p(\mathbf{x}^{(1)},\ldots,\mathbf{x}^{(L)}|\omega_k)p(\omega_k), \tag{2}$$

  with $k = 1,\ldots,c;\ k \neq j$.

Equation 2 provides the foundation for the development of five classifier combination rules: product rule, sum rule, minimum rule, maximum rule, and median rule.

Product Rule

- Assuming independence between the classes and vectors $\mathbf{x}^{(i)}$, with $i = 1, \ldots, L$, Equation 2 can be rewritten as:

$$\left(\prod_{i=1}^{L} p(\mathbf{x}^{(i)}|\omega_j)\right) p(\omega_j) > \left(\prod_{i=1}^{L} p(\mathbf{x}^{(i)}|\omega_k)\right) p(\omega_k), \tag{3}$$

  for $k = 1, \ldots, c;\ \ k \neq j$

- The left-hand side (or right-hand side) of the expression corresponds to:

$$\left(\frac{p(\omega_j|\mathbf{x}^{(1)})p(\mathbf{x}^{(1)})}{p(\omega_j)} \cdot \cdots \cdot \frac{p(\omega_j|\mathbf{x}^{(L)})p(\mathbf{x}^{(L)})}{p(\omega_j)}\right) p(\omega_j) =$$

$$= p(\omega_j)^{-L+1} \prod_{i=1}^{L} p(\omega_j|\mathbf{x}^{(i)}) \prod_{i=1}^{L} p(\mathbf{x}^{(i)}),$$

Product Rule (cont.1)

- Thus, we obtain the equivalence:

$$p(\omega_j)^{-L+1} \prod_{i=1}^{L} p(\omega_j|\mathbf{x}^{(i)}) \prod_{i=1}^{L} p(\mathbf{x}^{(i)}) > p(\omega_k)^{-L+1} \prod_{i=1}^{L} p(\omega_k|\mathbf{x}^{(i)}) \prod_{i=1}^{L} p(\mathbf{x}^{(i)}), \quad (4)$$

  for $k = 1, \ldots, L; k \neq j$.
- Assuming equiprobable classes – $p(\omega_j)^{-L+1} \equiv p(\omega_k)^{-L+1}$, $\forall j, k$ – and noting the presence of common terms on both sides of the inequality (which are also independent of the class occurrence), we arrive at...

Product Rule

$$(\mathbf{x}, \omega_j) \Leftrightarrow \arg\max_{\omega_j \in \Omega} \prod_{i=1}^{L} p(\mathbf{x}^{(i)}|\omega_j) \quad (5)$$

Sum Rule

- With a slightly stronger assumption:

$$p(\omega_j|\mathbf{x}_i) = p(\omega_j)(1+\epsilon_{ji}), \quad \text{such that} \quad \epsilon_{ji} \ll 1$$

- It assumes that the probability $p(\omega_j|\mathbf{x}_i)$ does not differ much from $p(\omega_j)$, with ϵ_{ji} accounting for the small observed deviations
- Applying this assumption to the left-hand side of the inequality in Equation 4 leads to...

Sum Rule

$$(\mathbf{x}, \omega_j) \Leftrightarrow \underset{\omega_j \in \Omega}{\arg\max} \sum_{i=1}^{L} p(\mathbf{x}^{(i)}|\omega_j) \cdot w_i \tag{6}$$

- The Sum Rule is more robust than the Product Rule in the presence of errors during the estimation of the *a posteriori* probabilities
- Moreover, it is possible to incorporate weights w_i into the Sum Rule
- An important aspect of this approach is tuning these weights, which can be determined through an additional training process aimed at minimizing the classification errors of the established combination

Maximum, Minimum, and Median Rules

Maximum Rule

$$(\mathbf{x}, \omega_j) \Leftrightarrow \max_{i=1,\ldots,L} \left\{ p(\omega_j|\mathbf{x}^{(i)}) \right\} > \max_{i=1,\ldots,L} \left\{ p(\omega_k|\mathbf{x}^{(i)}) \right\}; \; k = 1, \ldots, c; k \neq j \quad (7)$$

Minimum Rule

$$(\mathbf{x}, \omega_j) \Leftrightarrow \min_{i=1,\ldots,L} \left\{ p(\omega_j|\mathbf{x}^{(i)}) \right\} > \min_{i=1,\ldots,L} \left\{ p(\omega_k|\mathbf{x}^{(i)}) \right\}; \; k = 1, \ldots, c; k \neq j \quad (8)$$

Median Rule

$$(\mathbf{x}, \omega_j) \Leftrightarrow \text{med}_{i=1,\ldots,L} \left\{ p(\omega_j|\mathbf{x}^{(i)}) \right\} > \text{med}_{i=1,\ldots,L} \left\{ p(\omega_k|\mathbf{x}^{(i)}) \right\}; \; k = 1, \ldots, c; k \neq j \quad (9)$$

Agenda

1 Let's combine classifiers

2 Combination & Bayes

3 Majority Voting

4 Stacking

5 Bagging

6 AdaBoost/SAMME

7 Random Forest

Majority Voting

- Majority Voting represents a very intuitive and easy-to-implement classifier combination strategy
- The classification decision is made in favor of the class that receives the highest number of predictions from the constituent classifiers
- Assuming that the classifier is expressed in terms of prior probability, the following decision rule summarizes the discussed strategy:

$$(\mathbf{x}^{(i)}, \omega_j) \Leftrightarrow \sum_{i=1}^{L} \delta_{ji} > \sum_{i=1}^{L} \delta_{ki}; \;\; k = 1, \ldots, c; k \neq j, \tag{10}$$

where $\delta_{ji} = \begin{cases} 1, & \text{if } p(\omega_j|\mathbf{x}^{(i)}) = \max_i \left\{ p(\omega_\ell|\mathbf{x}^{(i)}) \right\}; \ell = 1, \ldots, c \\ 0, & \text{otherwise} \end{cases}$

Although the rule is based on prior probabilities, it can be easily extended to discriminant functions or other criteria

Agenda

1 Let's combine classifiers

2 Combination & Bayes

3 Majority Voting

4 Stacking

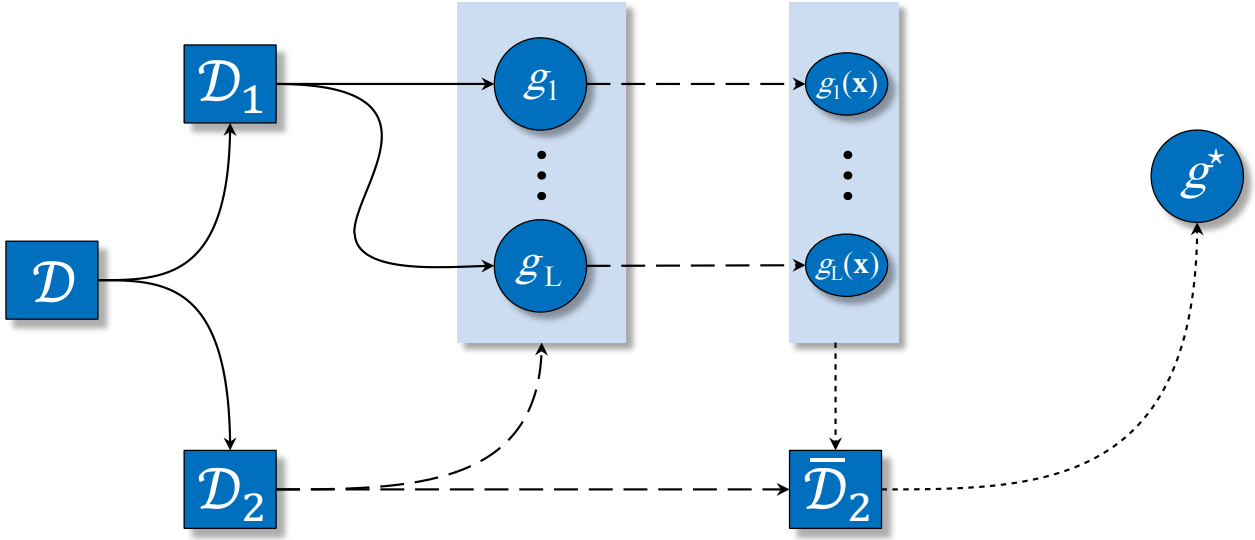
5 Bagging

6 AdaBoost/SAMME

7 Random Forest

Stacking

- Stacking aims to obtain a classifier that makes decisions based on the decisions previously obtained by a set of classifiers
- This process combines classifiers through the use of another classifier



Agenda

1 Let's combine classifiers

2 Combination & Bayes

3 Majority Voting

4 Stacking

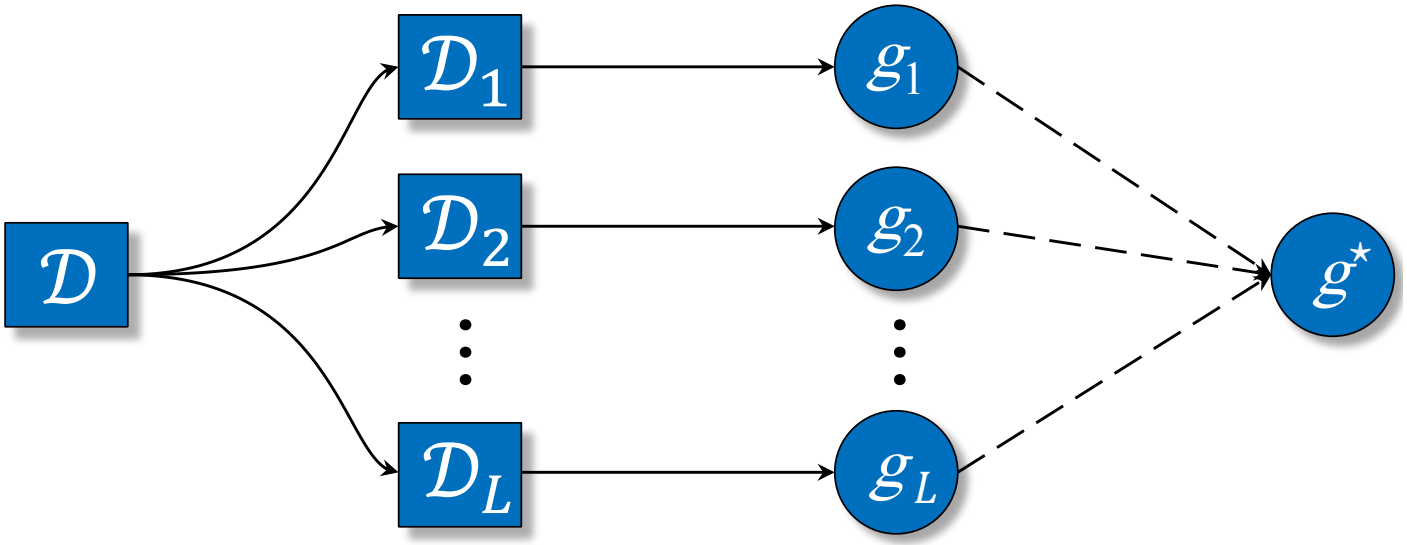
5 Bagging

6 AdaBoost/SAMME

7 Random Forest

Bagging

- Replicas of $\mathcal{D}$ are obtained through random sampling with replacement
- The number of elements in each replica is identical to the number of elements in $\mathcal{D}$
- In statistical literature, this procedure is known as bootstrap
- Thus, we can expect the classifiers to show some agreement among themselves, while also presenting divergences in more controversial cases
- Finally, disagreements among classifier decisions are resolved by Majority Voting



Agenda

1 Let's combine classifiers

2 Combination & Bayes

3 Majority Voting

4 Stacking

5 Bagging

6 AdaBoost/SAMME

7 Random Forest

AdaBoost/SAMME

- Trains a sequence of classifiers on the same dataset
- When a model misclassifies certain training examples, weights are applied to those examples so that the subsequent classifiers focus on them
- Given a training set $\mathcal{D} = \{(\mathbf{x}_i, y_i) \in \mathcal{X} \times \mathcal{Y} : i = 1, \ldots, m\}$, weights are initialized as $w_i = 1/m$, for $i = 1, \ldots, m$
- These weights, when considered over the original dataset, result in $\mathcal{D}_1$
- A sequence of L models is defined, denoted by g_j, for $j = 1, \ldots, L$

AdaBoost/SAMME (cont.1)

- Each g_j is trained on $\mathcal{D}_j$, for $j = 1, \ldots, L$
- For each training step, the following are computed: an error measure, a score, and a weight adjustment w_i to be used in the next training set $\mathcal{D}_{j+1}$

$$E_j = \sum_{i=1}^{m} w_i \left(1 - \delta\left(\widetilde{g}_j(\mathbf{x}_i) = y_i\right)\right) / \sum_{i=1}^{m} w_i \tag{11}$$

$$\lambda_j = \log\left(\frac{1 - E_j}{E_j}\right) + \log(c + 1) \tag{12}$$

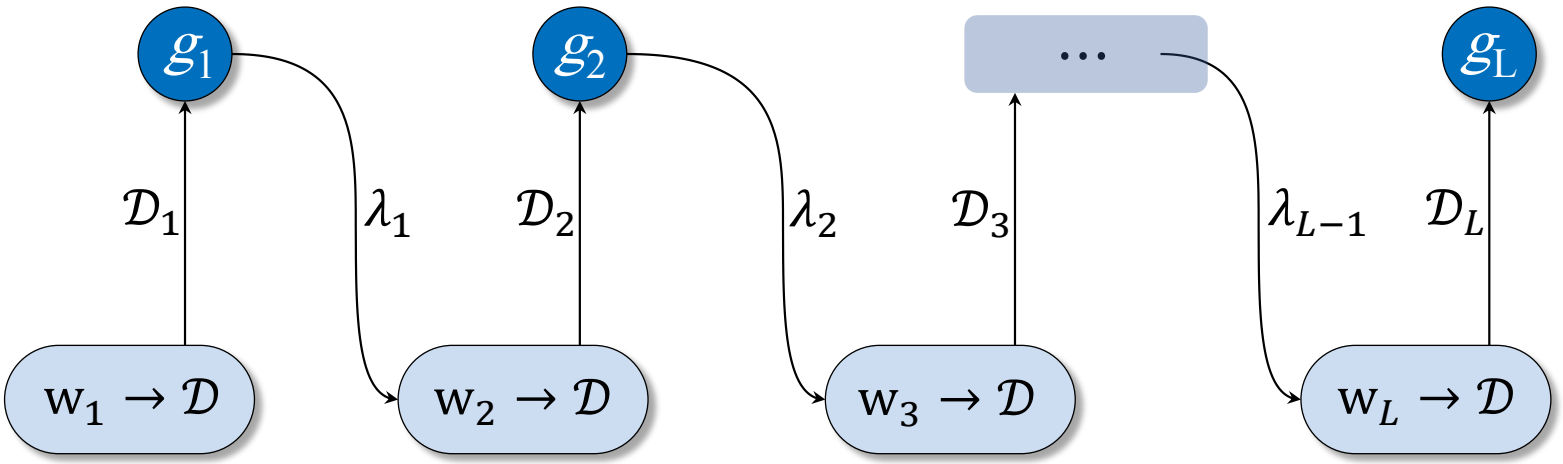
$$w_i := w_i e^{\lambda_j\left(1 - \delta\left(\widetilde{g}_j(\mathbf{x}_i) = y_i\right)\right)} \tag{13}$$

$$w_i := \frac{w_i}{\sum_{k=1}^{m} w_k} \tag{14}$$

AdaBoost/SAMME (cont.2)

At the end of the training of L models, the following decision rule is applied:

$$(\mathbf{x}, \omega_k) \Leftrightarrow \underset{k=1,\ldots,c}{\arg\max} \sum_{j=1}^{L} \lambda_j \delta\left(\widetilde{g}_j(\mathbf{x}_i) = y_i\right). \tag{15}$$



Agenda

1 Let's combine classifiers

2 Combination & Bayes

3 Majority Voting

4 Stacking

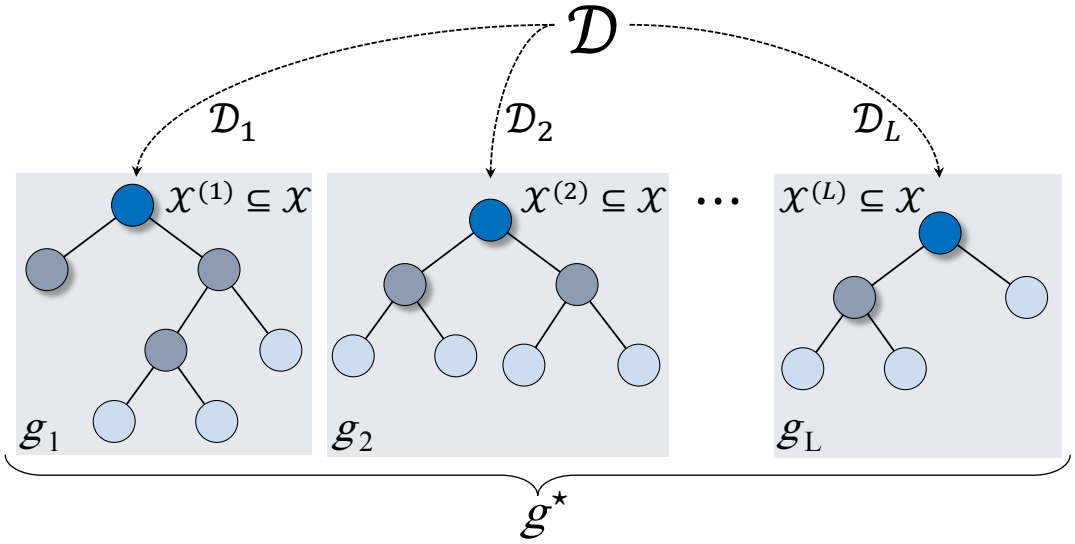
5 Bagging

6 AdaBoost/SAMME

7 Random Forest

Random Forest

- Random Forests comprise a combination of decision trees
- Two key aspects of this approach:
  - Use of *bootstrap* to generate training sets
  - Majority Voting to make the final decision
- Random Forests are almost a particular case of Bagging
- To further increase diversity among the trees, each one uses a random subset of the input attributes



Ensembles (Bayes, Voting, Bagging, Stacking, AdaBoost and Random Forest)

rogerio.negri@unesp.br